

Smart Guard — Part 1 explain

Contents

Part 1 — Code & Architecture Explanation	1
1. Goals (from the course brief)	2
2. System architecture (Part 1 slice)	2
3. Directory & file map (Part 1-relevant)	2
4. Process startup (main.c)	3
5. Configuration loading	4
6. HTTP → HTTPS redirect (detailed)	4
7. HTTPS / TLS server (detailed)	5
8. Self-signed certificate (scripts/gen_ssl.sh)	7
9. HTML dashboard (web/www/index.html)	8
10. Build system (web/Makefile)	9
11. systemd unit (services/web_server.service)	9
12. Concurrency model (why threads)	10
13. Data / paths used at runtime (Part 1)	10
14. End-to-end request walks	10
15. Dependencies (Part 1)	11
16. How Part 1 connects to later parts	11
17. Quick reference — Part 1 commands	11
18. PDF report package (what this part must contain)	11
19. Results analysis (Part 1)	12
20. Problems encountered and how they were solved (Part 1)	12
21. Conclusion	12
22. Related documents	13

Part 1 — Code & Architecture Explanation

Field	Value
Document	Final architecture & code explanation (submission)
Status	Complete — aligned with report/part 1/report.md
Companion	report/part 1/report.md (mandatory experiments 1-1 ... 1-6)
Also covers	PDF package: architecture, code, experiment tables/figures, results analysis, problems & solution
Project	Smart Guard System
Student	Parsa Nikookalam · 402102657
Scope	C web server, HTML dashboard, HTTPS (CN = student ID), HTTP→HTTPS redirect, systemd

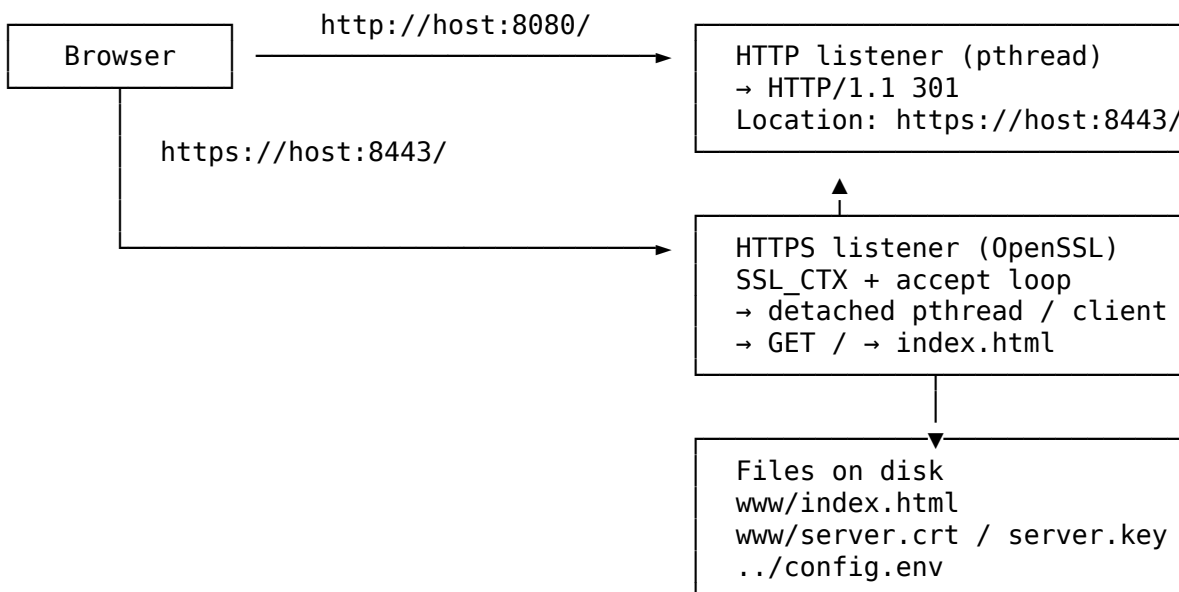
This document is the **finalized** description of Part 1 structure and source code. Later parts extend the same C process; Part 1 is the foundation.

1. Goals (from the course brief)

Requirement	How we meet it
Application core in C	Custom socket + OpenSSL server (web/web_server) — not Flask/Django/Node
HTML dashboard	web/www/index.html served on GET / over TLS
HTTPS + self-signed cert	server.crt / server.key; CN = STUDENT_ID
HTTP redirected to HTTPS	Separate listener returns 301 with Location: https://...
Auto-start after boot	services/web_server.service → Restart=always, enable

Platform note: On Orange Pi, ports are typically **80 / 443**. On WSL those ports are often reserved by Windows, so config.env uses **8080 / 8443**. Behaviour is identical; only the numbers change.

2. System architecture (Part 1 slice)



Boot: systemd → ExecStart=.../web/web_server (WorkingDirectory=web/)

Design principles:

1. **One binary** (web_server) owns both HTTP redirect and HTTPS serving.
2. **Config-driven ports** so the same code runs on WSL or Orange Pi.
3. **Threads, not fork-per-request**, for client handling (stable with later MQTT/email worker threads).
4. Vision / REST extras exist in the same tree but are **out of scope for Part 1 grading** — Part 1 only needs secure dashboard + systemd.

3. Directory & file map (Part 1-relevant)

```
embedded_project/
├── config.env                # STUDENT_ID, HTTP_PORT, HTTPS_PORT, TARGET
├── scripts/
│   ├── gen_ssl.sh           # openssl self-signed, CN=STUDENT_ID
│   └── install_services.sh / verify_* # install systemd units
```

services/	
└─ web_server.service	# autostart + Restart=always
web/	
└─ Makefile	# gcc → web_server
└─ web_server	# built binary
└─ include/	
└─ http_server.h	# void start_http_server(int port);
└─ src/	
└─ main.c	# process entry
└─ server.c	# sockets, TLS, redirect, HTML (core of Part 1)
└─ www/	
└─ index.html	# dashboard UI
└─ server.crt	# public certificate
└─ server.key	# private key (chmod 600)

File	Part 1 role
web/src/main.c	Starts background workers (later parts) then start_http_server()
web/src/server.c	All HTTP/HTTPS networking for Part 1 (https_client_thread, redirect thread)
web/www/index.html	Browser UI
web/include/http_server.h	Declares start_http_server()
scripts/gen_ssl.sh	Generates cert with correct CN + SAN
services/web_server.service	Boot persistence (Restart=always)
web/Makefile	Build recipe + OpenSSL / pthread / later-part libs

4. Process startup (main.c)

```
int main(void) {
    email_alert_start(); /* Part 3 – no-op path if EMAIL_ENABLED=0 */
    mqtt_pub_start();    /* Part 3 */
    part4_start();       /* Part 4 */
    start_http_server(0); /* Part 1 core – blocks in accept loops */
    return 0;
}
```

Why start other modules here? One systemd unit, one process. Part 1 still works if email/MQTT are disabled in config.env. The HTTP/TLS loop is what Part 1 requires.

start_http_server() (in server.c):

1. Loads ../config.env (relative to WorkingDirectory=web).
2. Spawns the **HTTP redirect** thread.
3. Creates **SSL_CTX**, loads www/server.crt + www/server.key.
4. Binds HTTPS socket and accepts clients forever.

4.1 Full main.c

```
/* web/src/main.c */
#include "http_server.h"
#include "email_alert.h"
#include "mqtt_pub.h"
#include "features_part4.h"

int main(void) {
    email_alert_start(); /* Part 3 – background SMTP thread */
```

```

mqtt_pub_start();      /* Part 3 – background MQTT thread */
part4_start();         /* Part 4 – Guard / watchdog / thermal */
start_http_server(0);  /* Part 1 – HTTP redirect + HTTPS (blocks) */
return 0;
}

```

Order matters: worker threads start **before** the blocking accept loops so email/MQTT/Part4 are alive while the server runs.

5. Configuration loading

Function: `load_config()` in `server.c`.

Reads line-oriented `KEY=VALUE` from `../config.env` (does **not** shell-source the file, so spaces in names are safer when quoted).

Key	Effect on Part 1
STUDENT_ID	Shown via later <code>/api/v1/config</code> ; used when generating cert
STUDENT_NAME	Dashboard identity
HTTP_PORT	Redirect listener (default 8080)
HTTPS_PORT	TLS listener (default 8443)
TARGET	ws1 vs board (affects reboot command later; not needed for plain Part 1 browse)

6. HTTP → HTTPS redirect (detailed)

6.1 Thread model

- `start_http_redirect_server()` runs on a **detached pthread**.
- Main thread remains free for HTTPS.
- Each incoming HTTP connection is also handled on a short-lived pthread (`http_redirect_thread`) so a slow client cannot block the accept loop.

6.2 Protocol behaviour

1. `read()` the request (enough to see headers).
2. Parse optional `Host:` header (strip port if present).
3. Reply:

```

HTTP/1.1 301 Moved Permanently
Location: https://<host>:<HTTPS_PORT>/
Connection: close

```

If `HTTPS_PORT == 443`, the Location omits the port (standard URL form).

6.3 Why Host-aware?

Using the `Host` header (instead of hard-coding `127.0.0.1`) keeps redirects correct when the student opens the page via LAN IP or hostname.

6.4 Source: HTTP 301 redirect (server.c)

```
/* http_redirect_thread - one client on the plain-HTTP port */
static void *http_redirect_thread(void *arg)
{
    int new_socket = *(int *)arg;
    free(arg);
    char buffer[BUFFER_SIZE] = {0};
    (void)read(new_socket, buffer, BUFFER_SIZE - 1);

    char host[128] = "127.0.0.1";
    char *h = strstr(buffer, "Host:");
    if (!h)
        h = strstr(buffer, "host:");
    if (h) {
        sscanf(h, "%*[^:]: %127s", host);
        char *colon = strchr(host, ':');
        if (colon)
            *colon = '\\0';
    }

    char response[320];
    /* WSL uses :8443 - include port in Location */
    snprintf(response, sizeof(response),
             "HTTP/1.1 301 Moved Permanently\r\n"
             "Location: https://%s:%d/\r\n"
             "Connection: close\r\n\r\n",
             host, g_https_port);
    (void)write(new_socket, response, strlen(response));
    close(new_socket);
    return NULL;
}
```

The accept loop for HTTP runs in `start_http_redirect_server()` on a detached pthread; each accepted socket gets its own `http_redirect_thread`.

6.5 Code anchors

Symbol	Location
<code>start_http_redirect_server</code>	<code>server.c</code>
<code>http_redirect_thread</code>	<code>server.c</code>

7. HTTPS / TLS server (detailed)

7.1 OpenSSL setup

Sequence inside `start_http_server()` (as in `server.c`):

1. `SSL_library_init()`, `OpenSSL_add_all_algorithms()`, `SSL_load_error_strings()`.
2. `SSL_CTX_new(TLS_server_method())`.
3. `SSL_CTX_use_certificate_file(ctx, "www/server.crt", SSL_FILETYPE_PEM)`.
4. `SSL_CTX_use_PrivateKey_file(ctx, "www/server.key", SSL_FILETYPE_PEM)`.
5. `bind / listen` on `HTTPS_PORT`.
6. Loop: `accept` → `allocate https_client_arg_t` → `detached https_client_thread` → `SSL_new / SSL_set_fd / SSL_accept` → `handle_https_client`.

Comment in code: *Do NOT fork() after mqtt/email/part4 threads start.*

7.2 Source: TLS accept + client thread (server.c)

```
/* Each HTTPS connection – detached pthread */
static void *https_client_thread(void *arg)
{
    https_client_arg_t *ca = (https_client_arg_t *)arg;
    int fd = ca->fd;
    SSL_CTX *ctx = ca->ctx;
    free(ca);

    SSL *ssl = SSL_new(ctx);
    if (!ssl) {
        close(fd);
        return NULL;
    }
    SSL_set_fd(ssl, fd);
    if (SSL_accept(ssl) > 0)
        handle_https_client(ssl); /* route GET /, APIs, stream, ... */
    SSL_shutdown(ssl);
    SSL_free(ssl);
    close(fd);
    return NULL;
}

void start_http_server(int ignore_port) {
    load_config();
    /* spawn HTTP redirect pthread ... */
    SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
    SSL_CTX_use_certificate_file(ctx, "www/server.crt", SSL_FILETYPE_PEM);
    SSL_CTX_use_PrivateKey_file(ctx, "www/server.key", SSL_FILETYPE_PEM);
    /* bind/listen on g_https_port ... */
    while (1) {
        new_socket = accept(server_fd, ...);
        https_client_arg_t *ca = malloc(sizeof(*ca));
        ca->fd = new_socket;
        ca->ctx = ctx;
        pthread_create(&th, &attr, https_client_thread, ca); /* detached */
    }
}
```

7.3 Source: send JSON/HTML helper

```
static void send_ssl_response(SSL *ssl, int status, const char *status_text,
                             const char *content_type, const char *body) {
    char header[512];
    int body_len = (int)strlen(body);
    int n = snprintf(header, sizeof(header),
                     "HTTP/1.1 %d %s\r\n"
                     "Content-Type: %s\r\n"
                     "Content-Length: %d\r\n"
                     "Connection: close\r\n"
                     "Access-Control-Allow-Origin: *\r\n"
                     "\r\n",
                     status, status_text, content_type, body_len);
```

```

    SSL_write(ssl, header, n);
    SSL_write(ssl, body, body_len);
}

```

7.4 Per-client worker (routing overview)

Each TLS session is handled on a **detached pthread**:

- Read full HTTP request (`ssl_read_http_request` — important later for POST bodies).
- Route by path (`path_match`).
- For Part 1's primary demo: missing API path → fall through to **serve `www/index.html`**.

7.5 Serving the HTML dashboard

Logic at the end of the HTTPS request handler:

1. `fopen("www/index.html", "r")` (path relative to `WorkingDirectory`).
2. Read entire file into memory.
3. Send headers:

```

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: <n>
Connection: close

```

4. `SSL_write` body; free buffer; close SSL/socket.

7.6 Source: serving `index.html`

```

/* End of handle_https_client - default route */
FILE *fp = fopen("www/index.html", "r");
/* ... fread entire file into file_buf ... */
char http_header[256];
snprintf(http_header, sizeof(http_header),
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/html; charset=utf-8\r\n"
    "Content-Length: %ld\r\n"
    "Connection: close\r\n\r\n",
    file_size);
SSL_write(ssl, http_header, hn);
SSL_write(ssl, file_buf, (int)file_size);

```

7.7 Why Connection: close?

Simplifies the custom server (no keep-alive / pipelining). Fine for a course appliance; browsers open new connections for assets/API polls.

8. Self-signed certificate (`scripts/gen_ssl.sh`)

8.1 What the script does

1. Reads `STUDENT_ID` from `config.env` **without** source (safe parsing).
2. Writes a temporary OpenSSL config with CN / O / C / SAN.
3. Runs `openssl req -x509 ....`
4. `chmod 600` on the private key.

8.2 Source: scripts/gen_ssl.sh (core)

```
STUDENT_ID="$(grep -E '^STUDENT_ID=' "$ROOT/config.env" | head -n1 \
| cut -d= -f2- | tr -d "'" | tr -d '"' | tr -d '\r')"

# openssl config fragment sets:
# CN = ${STUDENT_ID}
# O = Smart Guard System
# subjectAltName = DNS:localhost, IP:127.0.0.1, ...

openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout "$OUT/server.key" \
-out "$OUT/server.crt" \
-config "$CFG"

chmod 600 "$OUT/server.key"
openssl x509 -in "$OUT/server.crt" -noout -subject
```

8.3 Course requirement

Browsers will warn (self-signed). The **Certificate Viewer** must show **Common Name = student ID** — that is the Part 1 experiment evidence.

8.3 Files produced

File	Sensitivity
web/www/server.crt	Public — can be in the submission
web/www/server.key	Private — keep local; do not publish widely

9. HTML dashboard (web/www/index.html)

9.1 Role in Part 1

Part 1 only requires that a secure page is served. The HTML also contains hooks used by Parts 2-4 (telemetry cards, stream , Guard/Camera buttons). Serving richer HTML from the same C server is allowed: **logic remains in C**, UI is static/JS in the browser.

9.2 Typical browser behaviour

1. User opens `https://127.0.0.1:8443/`.
2. Browser TLS handshake with self-signed cert → warning → user proceeds.
3. C sends `index.html`.
4. Page JS may call `/api/v1/config` etc. (Part 2+); if those fail, the HTML shell still proves Part 1.

9.3 Stream image (later wiring)

The dashboard often embeds:

```

```

That URL is implemented in C as a proxy to the Python detector (:5000). For Part 1, the important fact is: **the page and stream URL are served by the C HTTPS process**, not by a Python web framework as the main server.

10. Build system (web/Makefile)

```
CC = gcc
CFLAGS = -Wall -Wextra -O2 -Iinclude -D_DEFAULT_SOURCE
LDFLAGS = -lssl -lcrypto -lsqlite3 -lcurl -lpthread -lmosquitto

SRC = src/main.c src/server.c src/telemetry.c ... # full project sources
TARGET = web_server
```

Flag / lib	Why
-lssl -lcrypto	TLS (Part 1)
-lpthread	Redirect + client threads
-lsqlite3	History/black box (Parts 2-4)
-lcurl	Email (Part 3)
-lmosquitto	MQTT (Part 3)

Build:

```
cd ~/embedded_project/web
make clean all
```

Produces ./web_server. systemd's WorkingDirectory must be ../web so relative paths (www/..., ../config.env) resolve.

11. systemd unit (services/web_server.service)

11.1 Important directives

Directive	Meaning
After= / Wants=	Prefer network (+ detector/MQTT when installed)
WorkingDirectory=	../web
EnvironmentFile=	Load config.env into the process environment
ExecStart=	Absolute path to web_server
Restart=always	Survive kill -9 (Part 1 experiment 1-2)
RestartSec=3	Brief backoff
WantedBy=multi-user.target	Enable on boot

11.2 Source: unit file excerpt (services/web_server.service)

```
[Unit]
Description=Smart Guard C Web Server (HTTP redirect + HTTPS REST)
After=network.target human_detector.service mosquitto_smartguard.service
```

```
[Service]
Type=simple
WorkingDirectory=/home/parsa/embedded_project/web
EnvironmentFile=/home/parsa/embedded_project/config.env
ExecStart=/home/parsa/embedded_project/web/web_server
Restart=always
RestartSec=3
```

```
[Install]
WantedBy=multi-user.target
```

11.3 Install pattern

```
sudo cp services/web_server.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now web_server
```

11.4 WSL autostart

WSL must run with systemd enabled ([boot] systemd=true in /etc/wsl.conf). Helper: scripts/enable_wsl_autostart.sh. Experiment **1-3** video uses wsl --shutdown then reopen Ubuntu without typing start commands.

12. Concurrency model (why threads)

Earlier prototypes used fork() per request. That conflicted with:

- Global OpenSSL / libcurl / mosquitto state
- Background MQTT and email threads

Current model: pthread per connection + dedicated background threads started from main. Long-lived MJPEG clients occupy one thread each without blocking telemetry or new accepts.

13. Data / paths used at runtime (Part 1)

Path	Use
web/www/index.html	Dashboard
web/www/server.crt	TLS cert
web/www/server.key	TLS key
../config.env	Ports + identity

No database is required for Part 1's core demo.

14. End-to-end request walks

14.1 First visit via HTTP

1. Browser → GET http://127.0.0.1:8080/
2. Redirect thread → 301 + Location: https://127.0.0.1:8443/
3. Browser follows → TLS handshake → GET /
4. C reads index.html → 200 HTML

14.2 Direct HTTPS

1. Browser → https://127.0.0.1:8443/
2. Warning (self-signed) → proceed
3. Same HTML path as above
4. Optional: open certificate details → CN = 402102657

14.3 After reboot

1. systemd starts web_server
 2. Both ports listening
 3. No manual ./web_server required
-

15. Dependencies (Part 1)

```
sudo apt install -y build-essential libssl-dev openssl
```

(Full project also needs curl, sqlite, mosquito, Python — see root README.md.)

16. How Part 1 connects to later parts

Later part	Reuses from Part 1
Part 2	Same HTTPS process adds REST routes in server.c
Part 3	Same process starts email/MQTT threads from main.c
Part 4	Same process runs Guard/watchdog/thermal thread

You do not replace the web stack in later parts — you extend it.

17. Quick reference — Part 1 commands

```
bash scripts/gen_ssl.sh
make -C web
sudo systemctl restart web_server
curl -v http://127.0.0.1:8080/ # expect 301
curl -sk -o /dev/null -w "%{http_code}\n" https://127.0.0.1:8443/ # expect 200
openssl x509 -in web/www/server.crt -noout -subject
systemctl status web_server --no-pager
```

18. PDF report package (what this part must contain)

Per the course PDF, the Part 1 submission package should include **architecture**, **code explanation**, **all experiment tables/figures**, **results analysis**, and **problems & how they were solved**. Mapping:

PDF expectation	Where it lives
Architecture	Sections 2–3 of this explain.md
Code explanation	Sections 4–12 of this explain.md (with source excerpts)
Experiment tables & charts/screenshots	report/part 1/report.md + files under fig/
Results analysis	Section 20 below + per-experiment verdicts in report.md
Problems & solutions	Section 21 below

18.1 Mandatory experiments — tables & figures checklist

No.	Experiment (PDF)	Required table / chart / media	File in fig/
1-1	Boot + systemd-analyze blame	Boot-time table + service share screenshot	01_boot_blame.png (op
1-2	kill -9 web server	journalctl auto-restart screenshot	02_kill9_restart.png
1-3	Power off / on once	Autostart video (no keyboard)	03_autostart.mp4
1-4	Open with http	Browser Inspect: 301 → HTTPS	04_http_301_inspect.p
1-5	Open self-signed cert	Cert details, CN = student ID	05_cert_subject.png
1-6	Open HTML page	Page with student ID visible	06_html_dashboard.png

Full write-up of each experiment (procedure + verdict): **report/part 1/report.md**.

19. Results analysis (Part 1)

Experiment	Expected outcome	Analysis
1-1	Blame table of boot cost	On WSL , boot is too fast for a meaningful Orange Pi-
1-2	Service comes back after SIGKILL	Restart=always in systemd replaces the PID; logs sh
1-3	App starts without typing start commands	After wsl --shutdown + reopen, web_server is already
1-4	HTTP → HTTPS with 301	Redirect thread parses Host and issues Location: h
1-5	CN = 402102657	gen_ssl.sh sets Distinguished Name CN to STUDENT
1-6	Dashboard shows student ID	HTML served over TLS; page loads config/ID for the c

Overall: Part 1 establishes a secure, auto-started C front-end. The only platform exception is **1-1** (WSL boot), which is explicitly documented and covered by **1-3**.

20. Problems encountered and how they were solved (Part 1)

#	Problem	Severity	Cause
1	Ports 80/443 unavailable on WSL	High	Windows reserves privile
2	fork() per HTTPS request crashed with background workers	High	Fork after MQTT/email th
3	Browser warns on certificate	Low (expected)	Self-signed cert (course
4	WSL does not behave like board cold boot	Medium (exp 1-1)	Lightweight VM start
5	Relative paths fail if cwd wrong	Medium	Cert/HTML looked up as
6	config.env name with spaces broke source	Medium	Unquoted STUDENT_NAM

21. Conclusion

Part 1 delivers a **minimal embedded HTTPS appliance** in C:

Property	Implementation
Dual listeners	HTTP 301 redirect + OpenSSL HTTPS
Identity	Self-signed certificate, CN = 402102657
UI	www/index.html served on GET /

Property	Implementation
Resilience	Restart=always after kill -9 (exp 1-2)
Persistence	web_server.service enable-on-boot; WSL autostart proven in exp 1-3 video

1-1 is **N/A on WSL** (boot too short for a board-style systemd-analyze blame table); autostart evidence is redirected to **1-3**. Experiments **1-2 ... 1-6** are complete with media under fig/.

All later Smart Guard features run inside this same process and certificate boundary.

22. Related documents

Document	Role
report/part 1/report.md	Mandatory experiments 1-1 ... 1-6 (final)
report/part 1/explain.md	This file — architecture & code (final)
README.md (repo root)	Full-project setup guide